```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm, datasets
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
from sklearn.metrics import plot_confusion_matrix
from mlxtend.plotting import plot_decision_regions
```

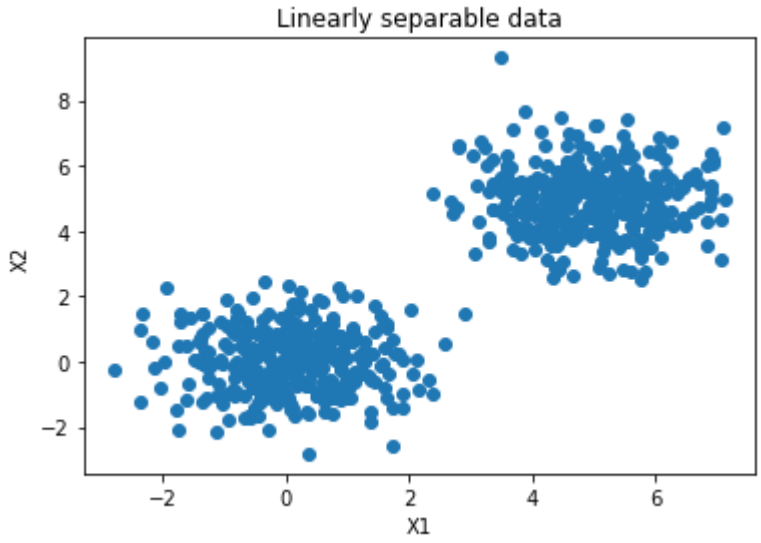
Exemple 1

Construction de la base Blob

```
blobs_random_seed = 42
centers = [(0,0), (5,5)]
cluster_std = 1
frac_test_split = 0.33
num_features_for_samples = 2
num_samples_total = 1000
inputs, targets = make_blobs(n_samples = num_samples_total, centers = centers, n_features = n
X_train, X_test, y_train, y_test = train_test_split(inputs, targets, test_size=frac_test_spli
```

Visualisation de la base

```
# Generate scatter plot for training data
plt.scatter(X_train[:,0], X_train[:,1])
plt.title('Linearly separable data')
plt.xlabel('X1')
plt.ylabel('X2')
plt.show()
```



```
lin_svc = svm.SVC(kernel='linear',C=1)
lin_svc.fit(X_train, y_train)
```

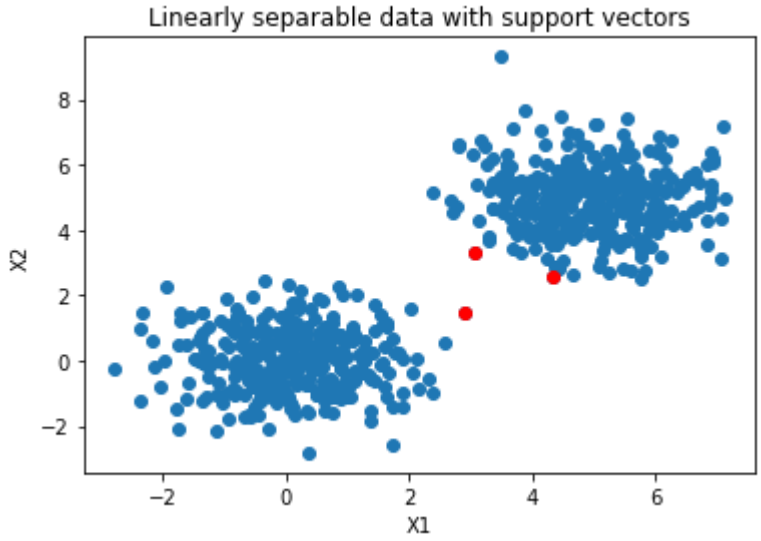
```
SVC(C=1, kernel='linear')
```

```
lin_svc.score(X_test, y_test)
```

```
1.0
```

Visualisation des vecteurs de support

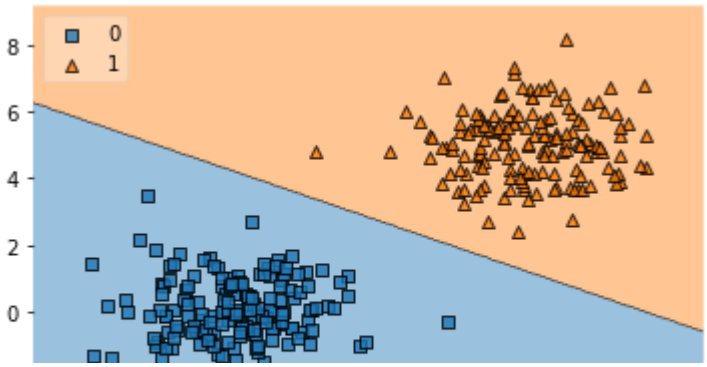
```
# Get support vectors
support_vectors = lin_svc.support_vectors_

# Visualize support vectors
plt.scatter(X_train[:,0], X_train[:,1])
plt.scatter(support_vectors[:,0], support_vectors[:,1], color='red')
plt.title('Linearly separable data with support vectors')
plt.xlabel('X1')
plt.ylabel('X2')
plt.show()
```



Visualusation de la surfece de décision

```
plot_decision_regions(X_test, y_test, clf=lin_svc, legend=2)
plt.show()
```

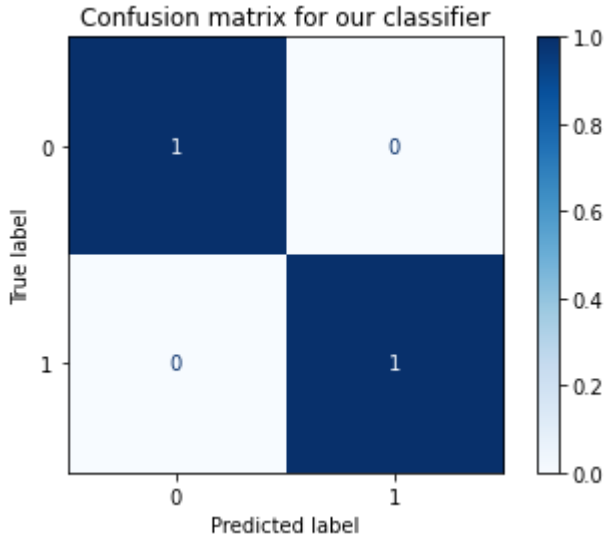
```
/usr/local/lib/python3.7/dist-packages/mlxtend/plotting/decision_regions.py:244: Matplot
  ax.axis(xmin=xx.min(), xmax=xx.max(), y_min=yy.min(), y_max=yy.max())
```



Matrice de confusion


```
# Predict the test set
predictions = lin_svc.predict(X_test)

# Generate confusion matrix
matrix = plot_confusion_matrix(lin_svc, X_test, y_test,
                               cmap=plt.cm.Blues,
                               normalize='true')
plt.title('Confusion matrix for our classifier')
plt.show(matrix)
plt.show()
```

```
/usr/local/lib/python3.7/dist-packages/sklearn/utils/deprecation.py:87: FutureWarning: F
  warnings.warn(msg, category=FutureWarning)
```



Exemple 2

Charger Iris dataset et ne garder que les deux premiers attributs

```
iris = datasets.load_iris()
X, y = iris.data[:, :2], iris.target
```

Préparer les bases d'apprentissage et de test

```
# On conserve 50% du jeu de données pour l'évaluation
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

Phase d'pprentissage avec SVM Linéaire

```
C = 1.0 # paramètre de régularisation
lin_svc = svm.LinearSVC(C=C)
lin_svc.fit(X_train, y_train)
```

```
/usr/local/lib/python3.7/dist-packages/sklearn/svm/_base.py:1208: ConvergenceWarning: Li
  ConvergenceWarning,
LinearSVC()
```

Phase du test

```
lin_svc.score(X_test, y_test)
```

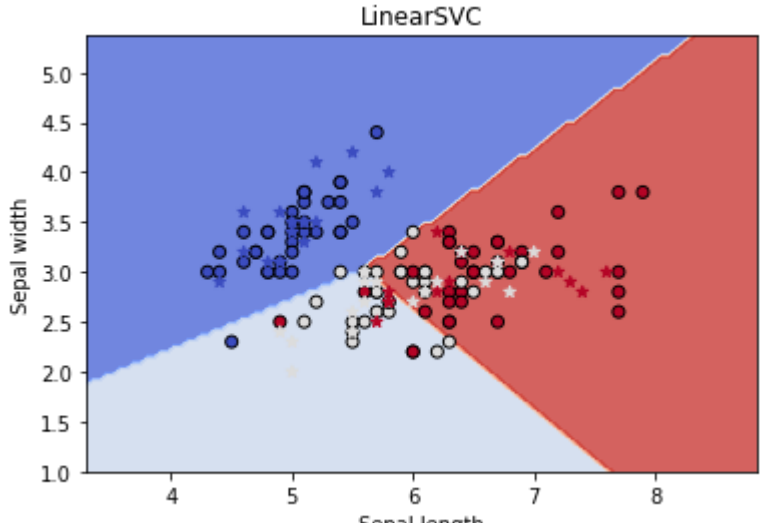
```
0.7111111111111111
```

Visualisation de la surface de décision

```
# Créer la surface de décision discretisée
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
# Pour afficher la surface de décision on va discrétiser l'espace avec un pas h
h = max((x_max - x_min) / 100, (y_max - y_min) / 100)
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

# Surface de décision
Z = lin_svc.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.8)
# Afficher aussi les points d'apprentissage
plt.scatter(X_train[:, 0], X_train[:, 1], label="train", edgecolors='k', c=y_train, cmap=plt.
plt.scatter(X_test[:, 0], X_test[:, 1], label="test", marker='*', c=y_test, cmap=plt.cm.coolw
plt.xlabel('Sepal length')
```

```
plt.ylabel('Sepal width')
plt.title("LinearSVC")
```

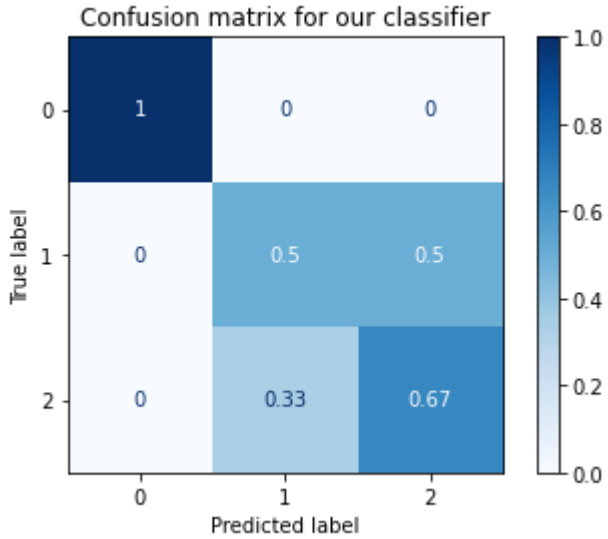
```
Text(0.5, 1.0, 'LinearSVC')
```



Matrice de confusion

```
# Generate confusion matrix
matrix = plot_confusion_matrix(lin_svc, X_test, y_test,
                               cmap=plt.cm.Blues,
                               normalize='true')
plt.title('Confusion matrix for our classifier')
plt.show(matrix)
plt.show()
```

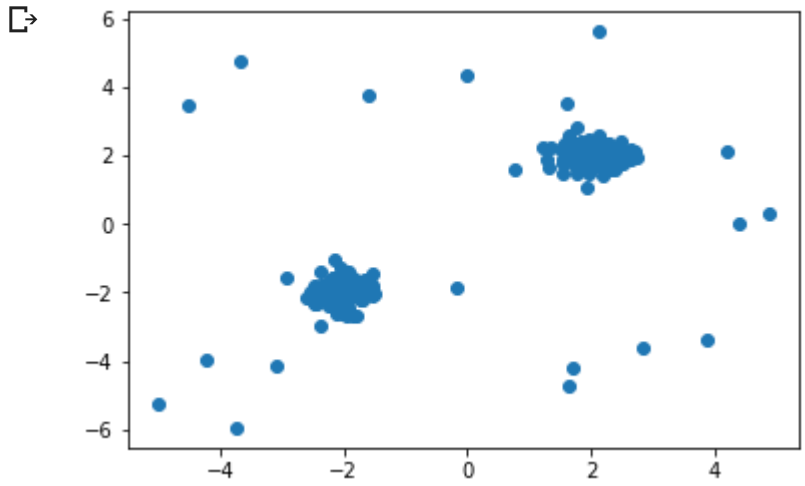
```
/usr/local/lib/python3.7/dist-packages/sklearn/utils/deprecation.py:87: FutureWarning: F
  warnings.warn(msg, category=FutureWarning)
```



Exemple 3

Construction de la base

```
N = 200
data1·=·0.3·*·np.random.randn(N·//·2,·2)·+·[2,2]
data2·=·0.3·*·np.random.randn(N // 2, 2) - [2,2]

# On créé 10% de données anormales (*outliers*)
outliers = np.random.uniform(size=(N // 10, 2), low=-6, high=6)

# Les données = groupes + anomalies
X = np.concatenate((data1, data2, outliers))

plt.scatter(X[:,0], X[:,1]) and plt.show()
```



Construction du modèle

```
clf = svm.OneClassSVM(nu=0.1, kernel="rbf", gamma=0.05)
clf.fit(X)
```

```
     OneClassSVM(gamma=0.05, nu=0.1)
```

```
xx, yy = np.meshgrid(np.linspace(-7, 7, 500), np.linspace(-7, 7, 500))
Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
y_pred = clf.predict(X)

# Choix du jeu de couleurs
plt.set_cmap(plt.cm.Paired)
# Trace le contour de la fonction de décision
plt.contourf(xx, yy, Z)
# Affiche les points considérés comme "inliers"
plt.scatter(X[y_pred>0,0], X[y_pred>0,1], c='white', edgecolors='k', label='inliers')
# Affiche les points considérés comme "outliers"
```

```
plt.scatter(X[y_pred<=0,0], X[y_pred<=0,1], c='black', label='outliers')
plt.legend()
plt.show()
```